הנחיות מפורטות יותר לתרגיל 9

בעקבות מלא שאלות וחוסר בהירות סביב התרגיל, שהוא יחסית תרגיל פשוט למימוש, אלא שההוראות שניתנו עבורו מעומעמות מאוד. ניסיתי לאסוף עבורכם מהפרום ומשיחות עם אנשים את אוסף ההנחיות וההוראות המפורטות יותר לביצוע התרגיל.

אם למישהו יש הערות ונקודות שהוא חושב שטעיתי בהן, מוזמן בכיף לעדכן.

הנחיות מפורטות למימוש

על מנת להתחיל, יש להתחיל במימוש פשוט של המחלקה car

המחלקה "**car**"

1. זוהי מחלקה שאמורה לתפקד באופן עצמאי. כלומר: אין מגבלות על הצבע של הרכב, המיקום שלו בקוואורדינטות או שם הרכב.
2. תכונות- התכונות שניתנו לנו בתרגיל, לא מסובך.
3. הפעולה: **car_coordinates** : כאמור, אין שום הגבלה על הקוואורדינטות בהן יימצא הרכב. זכרו: להחזיר את הקוואורדינטות כlist של tuples.
4. הפעולה: **possible_moves**: פעולה פשוטה של החזרת התנועות האפשריות של כל רכב באשר הוא. לא צריך לבצע בדיקת תקינות כי הרכב כבר קיים ועבר את הבדיקות הנחוצות, הפעולה בסך הכול מציגה את התנועות האפשריות שלו.
5. הפעולה: **movement_requirements** עבור כל תנועה יש משבצת אחת שצריכה להיבדק. עבור הזזה אנכית למעלה- המשבצת מעל ל **location** של המכונית, עבור הזזה אנכית למטה- המשבצת מתחת ל"זנב" של המכונית (המיקום ההפוך ל **location**), עבור הזזה אופקית לצד ימין, יש לבצע בדיקה למשבצת אחת ימינה מה"זנב" של המכונית, ועבור הזזה שמאלה, בדיקה למשבצת משמאל לראש המכונית. ניתן לבצע בדיקה שהתו שהוזן לבדיקת התנועה הוא תו נכון. יש לזכור להחזיר טאפל בתוך ליסט!
6. הפעולה **move** כאן, יש לבצע בדיקת תקינות של הקלט 'movekey' , בדיקה שהאוריינטציה של המכונית מתאימה לפקודה שניתנה לה. כלומר, מכונית לא תוכל לזוז למעלה אם האוריינטציה של היא אופקית (1). חשוב לזכור כי הפעולה מזיזה את המכונית בפועל לקוואורדינטות חדשות ולא רק מחזירה משתנה בוליאני שמסמל האם ניתן להזיז את המכונית או לא!
7. קבועים אפשריים: 1.האוריינטציה של המכונית

כעת, יש לכם רכב ואתם יכולים לממש את המחלקה **board**

המחלקה "**board**"

1. קונסטקטור: בשביל לממש את המלקה הנ"ל, מומלץ לממש לפחות 2 פרמטרים כתכונות פנימיות של הלוח בקונסטרקטור של המחלקה: א. ייצוג גראפי של הלוח על ידי רשימה של רשימות שבתוכן התאים הריקים מיוצגים ב"_". לאחר מכן יש להוסיף עמודה של כובביות שמקום ה (3,7) יש תא המייצג את היציאה, לדוגמה באמצעות E. ניתן לממש את הבנייה של הלוח כפעולה פנימית ולאתחל איתה את התכונה של הייצוג הגראפי בכל בנייה של לוח. ב. רשימה או מילון של כל המכוניות הנמצאות על הלוח. הרשימה או המילון יכללו בתוכן טיפוסים מסוג car.

2. **__str__** אפשר להתשמש בפונקציית join לייצוג דומה לדוגמה המוצגת בהוראות התרגיל.
3. **possible_moves** בפונקציה הזו, יש לעבור על כל הרכבים שנמצאים ברשימת הרכבים בלוח, אותה הגדרתם בבניית הלוח, ולהדוק אלו תנועות אפשריות יש לכל מכונית. כאן, תנועה אפשרית רק אם אין מכונית שמגבילה את התנועה של המכונית לכיוון מסוים, והמכונית הנתונה לא חורגת מגבולות הלוח. יש לזכור כי תנועה למשבצת היציאה מוגדרת כתנועה אפשרית, על אף שהיא מחוץ לריבוע הראשי. ניתן להתשמש במימוש של possible_moves במחלקה car על מנת לממש את הפעולה הנ"ל. להחזיר רשימה של טאפלים!
4. הפעולה **cell_content** תשימו לב טוב באיזה מאפיין של הלוח אתם בודקים את התכולה של התא באותו מקום אותו אתם נדרשים לבדוק!
5. הפעולה **add_car**: כאן, חשוב לזכור כי יש לבצע כמה בדיקות קודם הכנסת רכב ללוח שתי בדיקות שקשורות ללוח עצמו: **1.** שאין עוד רכב עם אותו שם בלוח (הלוח לא מוגבל לצבעים מסויימים, והבדיקה הנכונה היא רק כלפי השמות) 2. בדיקה שהרכב שמוסיפים לא חורג מגבולות הלוח 3. הרכב לא מתווסף על קווארדינטות תפוסות. ההוספה עצמה: תתבצע בשני מישורים: 1. ברשימת המכוניות של הלוח 2. בייצוג הגראפי של המכונית על הלוח. בנוסף יש להחזיר משתנה בוליאני שמתאר אם ההוספה הצליחה או לא.
6. הפונקציה **move car** ניתן לעשות בדיקות על הפרמטרים בפונקציה, כלומר 1. אם הקלט של הכיוון תקין ו2. לבדוק אם קיים רכב מאותו שם ברשימת המכוניות שעל הלוח.
לפני התזוזה יש לבדוק: 1. שאין מכוניות באותו מקום אליו המכונית מעויינת לזוז 2. שאין חריגה מגובלות הלוח.
לאחר מכן- על ידי שמירה של הקווארדינטות ועדכון מתאים של 1. רשימת המכוניות 2. הייצוג הגראפי של הלוח, ניתן לבצע את ההזזה ולהחזיר ערך בוליאני מתאים. חשוב לזכור שאם מזיזים רכב לכיוון מסויים, הוא זז לקווארדינטה חדשה אך במקביל מפנה ללוח "_" קווארדינטה חדשה בהתאם לכיוון אליו זז.
7. קבועים אפשריים: 1. מספר העמודות 2. מספר השורות 3. קווארדינטת יציאה. חשוב לזכור כי על פי ההוראות קיים לוח משחק אחד אופציונאלי, אותו לוח משחק הוא לא יכול להיות מאותחל בgame שמקבל כבר את הלוח בתור תכונה.

עכשיו, אתם יכולים סוף סוף להגיע לשלב של יצירת משחק עם חוקים והרצה, על ידי מימוש המחלקה השלישית והאחרונה:

המחלקה "**game**"

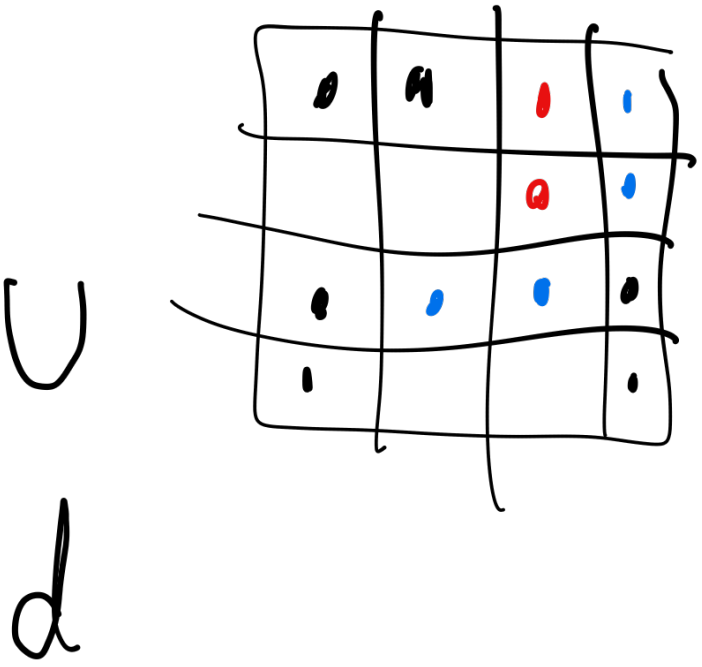
1. בראש הקובץ, יש לייבא את מחלקות : **car, board, game,sys_** ובנוסף את ה **helper** שניתן לנו בקבצים הנוספים.
2. קונסטרקוור: בתכונות של המשחק יש טיפוס מסוג board אותו תבנו אח"כ ב- **main**. ניתן לממש את המשחק רק עם תכונה זו.
3. הפעולה **single turn**: הפעולה מריצה תור אחד כלומר תזוזה אחת של רכב במשחק. יש לבצע בה את הבדיקות הבאות על הקלט: 1. מפתח התזוזה תקין. 2. קיימת עוד אפשרות לתנועה על לוח המשחק. 3. המשחק עוד לא הסתיים. המשחק מסתיים כאשר הוקלד הסימן ! או כאשר קיימת מכונית שהגיעה לקווארדינטת היציאה. 4. שם הרכב תקין.5. הקלט עצמו תקין. כאן יודפסו כל הודעות השגיאה בהתאם לבעיה בה נתקל המשחק. ניתן להתחשב בהחזרות בוליאניות מהמחלקות השונות **car** ו- **board.**

4. <u>הפעולה **play::**</u> גם כאן יש לוודא: כי ישנן תנועות אפשריות עוד על הלוח ושהמשחק לא נגמר. ניתן להדפיס הודע המשקפת את מצב השחקן בסוף המשחק.

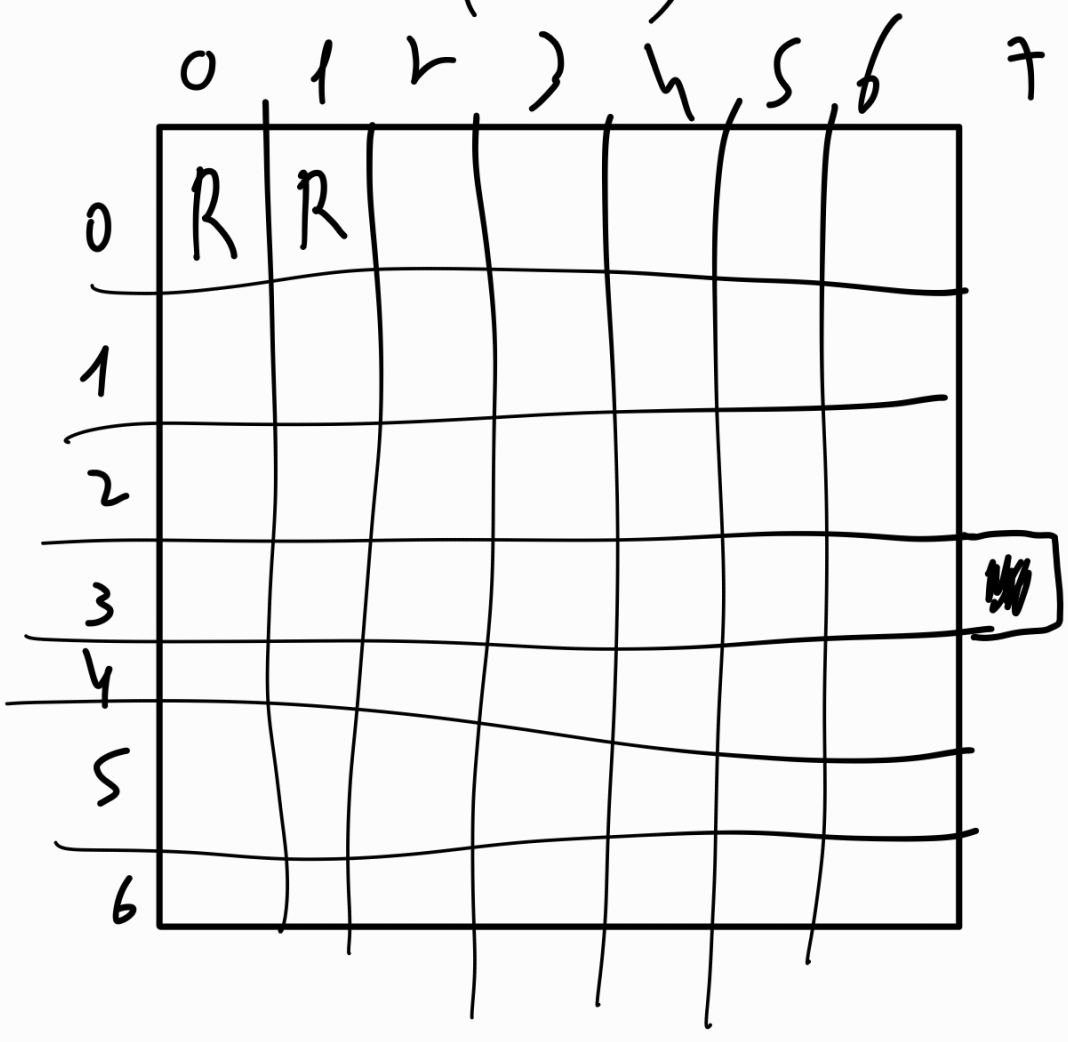
כעת, אנחנו מגיעים לmain

<u>הmain</u>

1. יצירת לוח באמצעות הקונסטרקטור של הלוח אותו הגדרנו כבר במחלקה
2. באמצעות הפעולה sys.argv שתוכנס לתוך משתנה אותו נגדיר, נקבל את קובץ הנתונים.
3. כעת, נטען את כל המכוניות לתוך מילון אותו נגדיר. את המילון נקבל באמצעות הפעלת פונקציית הhleper על המקום ה- 1 (לא ה- 0) ב- argv.
4. כעת נבצע הכנסה של מכוניות ללוח בצורה הבאה: א. בדיקה שצבע המכונית תקין ושהשם שלה באורך הנכון ב. בדיקה שהמכונית באורך הנכון, כלומר בין 2 ל4 משבצות. ג. לא צריך לבצע כמובן בדיקה אם קיימות שתי מכוניות באותו השם כי המילון יטען כבר באופן אוטומטי בערך יחיד. ד. יצירת אובייקט car מהערך שעבר את כל הבדיקות והוספתו ללוח באמצעות add car
5. יצירת אובייקט מסוג משחק והגדרת הלוח שבנינו והטענו במכוניות בתור תכונת הלוח של המשחק.
6. הפעלת הפונקצייה play על אובייקט המשחק.



(1, 4)



Pre = (3, 4), (3, 5)

Post = (3, 5), (3, 6)

Post

∂n

$\emptyset n$

n^2

$\alpha n)$

r: movie

o: name

['p', 'r']

[